

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
INSTITUTO METRÓPOLE DIGITAL

IMD0039 — Estruturas de Dados Básicas II

Análise Empírica de Algoritmos

O impacto da estrutura de dados da fila de prioridade
no tempo de execução do algoritmo de Dijkstra

Trabalho da Unidade 1

Integrantes do grupo

Luiz Victor	[PREENCHER: matrícula]
[PREENCHER: nome]	[PREENCHER: matrícula]
[PREENCHER: nome]	[PREENCHER: matrícula]
[PREENCHER: nome]	[PREENCHER: matrícula]

Profa. Ligia Borges

Currais Novos / Natal — RN
8 de setembro de 2026

Sumário

1	Introdução	1
2	Algoritmos e fontes	1
2.1	Como as fontes foram localizadas	1
2.2	Fonte principal	2
2.3	Algoritmos selecionados	2
3	Metodologia	3
3.1	Definição do tamanho da entrada n	3
3.2	Geração das entradas	3
3.3	Adaptações feitas no código	4
3.4	Validação	4
3.5	Metodologia de medição	5
4	Análise teórica da complexidade	5
4.1	Dijkstra Normal — $O(n^2)$	6
4.2	Dijkstra com Heap Mínimo — $O((n + m) \log n)$	6
4.3	Dijkstra com Buckets — $O(m + n + D_{max})$	7
5	Resultados	8
5.1	Tempos medidos	8
5.2	Comparação com a curva de crescimento esperada	9
5.3	Grandezas estruturais coletadas	11
6	Discussão: teoria $\times$ prática	11
6.1	O crescimento observado é compatível com a complexidade teórica?	11
6.2	Que fatores explicam o desvio do Dijkstra Normal?	12
6.3	Em que ponto a diferença entre os algoritmos fica evidente?	13
6.4	O que acontece quando n aumenta muito?	13
7	Conclusão	14
	Declaração de uso de ferramentas de IA	15
	Referências	15
A	Instruções de execução	17
B	Código-fonte	18

1 Introdução

O problema do caminho mínimo consiste em determinar, em um grafo com pesos não negativos nas arestas, o menor custo para ir de um vértice de origem até todos os demais vértices. É um dos problemas mais aplicados da computação: aparece em sistemas de navegação, em roteamento de pacotes em redes, em planejamento logístico e em problemas de otimização em geral. O algoritmo clássico para resolvê-lo é o algoritmo de Dijkstra, publicado em 1959 (DIJKSTRA, 1959).

O ponto que motiva este trabalho é que o algoritmo de Dijkstra não é um único algoritmo, e sim um *esqueleto*: ele repete o par de operações “extrair o vértice aberto de menor distância estimada” e “relaxar as arestas desse vértice” até fechar todos os vértices. A ordem em que os vértices são visitados e o resultado final são sempre os mesmos, mas o **custo** da operação de extração do mínimo depende inteiramente da estrutura de dados usada para manter a fila de prioridade. Isso faz do Dijkstra um caso quase ideal para uma investigação empírica: é possível manter fixos o problema, a entrada, a linguagem e o compilador, e variar apenas a estrutura de dados.

Este relatório investiga, portanto, a seguinte pergunta:

Como o tempo de execução do algoritmo de Dijkstra cresce em função do número de vértices do grafo, e de que forma esse crescimento muda quando a fila de prioridade é implementada por varredura linear de vetor, por heap mínimo e por buckets?

Foram implementadas e medidas três variantes: o **Dijkstra Normal** (seleção do mínimo por varredura linear, $O(n^2)$), o **Dijkstra com Heap Mínimo** (fila de prioridade em *heap* binário, $O((n + m) \log n)$) e o **Dijkstra com Buckets**, na formulação de Dial (fila de prioridade monótona com *buckets* circulares, $O(m + n + D_{max})$). O objetivo do experimento é verificar se o crescimento observado é compatível com essas previsões assintóticas e, onde houver divergência, identificar a causa.

2 Algoritmos e fontes

2.1 Como as fontes foram localizadas

A busca partiu do tema “caminho mínimo em grafos” com foco em trabalhos que comparassem estruturas de dados de forma experimental.

- **Palavras-chave utilizadas:** **[PREENCHER: palavras-chave que o grupo realmente digitou, por exemplo “algoritmo de Dijkstra estruturas de dados desempenho”]**

- Base/local de busca: [PREENCHER: onde o grupo buscou — por exemplo Google Acadêmico, repositório institucional da UFES, Portal de Periódicos da CAPES]
- Data de acesso: [PREENCHER: data]

2.2 Fonte principal

RODRIGUES, Guilherme Conrado da Silva. **Análise de desempenho do algoritmo de Dijkstra com diferentes estruturas de dados**. 2023. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação) — Departamento de Computação, Centro de Ciências Exatas, Naturais e da Saúde, Universidade Federal do Espírito Santo, Alegre, ES, 2023. Orientador: Prof. Dr. Edmar Hell Kampke. Aprovado em 15 de dezembro de 2023.

Repositório de código indicado pela fonte: <https://github.com/guiconrado/Dijkstra>

Esse TCC foi escolhido como fonte principal por três razões: (i) apresenta os pseudo-códigos completos das três variantes que queríamos comparar; (ii) informa explicitamente a complexidade esperada de cada uma; e (iii) já executa uma comparação de desempenho, o que nos dá um resultado de referência contra o qual confrontar as nossas medições.

O trabalho de referência mediu as três variantes sobre quatro instâncias reais de redes de estradas dos Estados Unidos, obtidas do *9th DIMACS Implementation Challenge — Shortest Paths*, e obteve tempos médios de 407,242 s para o Dijkstra Normal, 0,032 s para o Heap Mínimo e 0,184 s para o Bucket, concluindo que o Heap Mínimo foi cerca de seis vezes mais rápido que o Bucket naquele conjunto de testes.

2.3 Algoritmos selecionados

Tabela 1: Algoritmos comparados neste trabalho.

Algoritmo		Estrutura da fila de prioridade	Complexidade indicada pela fonte	Contexto de aplicação
Dijkstra	Normal	Vetor de distâncias com varredura linear	$O(n^2)$	Grafos densos ou pequenos; implementação de referência, usada em ensino
Dijkstra	Heap Mínimo	<i>Heap</i> binário (fila de prioridade)	$O((n + m) \log n)$	Uso geral; padrão em bibliotecas e em roteamento em redes esparsas
Dijkstra	Bucket	<i>Buckets</i> circulares (Dial)	$O(m + n + D_{max})$	Grafos com pesos inteiros de intervalo pequeno, como redes de estradas

As três variantes resolvem exatamente o mesmo problema (caminho mínimo de origem única com pesos não negativos) e produzem a mesma saída, o que torna a comparação legítima. A estrutura de *buckets* tem origem em Dial (1969) e é descrita tanto pela fonte principal quanto por Cormen *et al.* (2009).

3 Metodologia

3.1 Definição do tamanho da entrada n

Neste experimento, n é o número de vértices do grafo.

Essa escolha exige um cuidado: o tempo do Dijkstra depende de duas grandezas, o número de vértices n e o número de arestas m . Se as duas variassem livremente, não seria possível atribuir o crescimento observado a uma delas. Por isso fixamos a **densidade** do grafo em grau médio 8, ou seja, $m = 4n$ arestas não direcionadas. Com isso m cresce linearmente com n e os grafos permanecem esparsos em toda a faixa testada, que é justamente o regime em que as três complexidades teóricas se separam:

$$\text{Normal: } O(n^2 + m) = O(n^2)$$

$$\text{Heap: } O((n + m) \log n) = O(n \log n)$$

$$\text{Bucket: } O(m + n + D_{\max}) = O(n + D_{\max})$$

3.2 Geração das entradas

O trabalho de referência usou instâncias fixas do DIMACS. Essas instâncias são excelentes por serem redes reais, mas têm um tamanho fixo cada uma, o que não permite variar n de forma controlada — e variar n é exatamente o objetivo deste trabalho. Substituímos então as instâncias fixas por um gerador paramétrico:

1. sorteia-se uma árvore geradora aleatória sobre os n vértices (cada vértice i é ligado a um vértice sorteado entre 0 e $i - 1$), o que **garante que o grafo seja conexo** e portanto que todos os n vértices sejam alcançados em toda execução;
2. completa-se com arestas aleatórias até atingir $m = 4n$;
3. cada aresta recebe um peso inteiro sorteado uniformemente em $[1, C]$, com $C = 1000$.

O gerador usa `mt19937_64` com semente determinística em função de n e do número da repetição, de modo que **todo o experimento é reprodutível** e, para um mesmo par $(n, \text{repetição})$, as três variantes recebem exatamente o mesmo grafo. O vértice de origem é sempre o vértice 0.

3.3 Adaptações feitas no código

Partindo dos pseudocódigos da fonte, as seguintes alterações foram necessárias:

1. **Leitura de arquivo substituída pelo gerador paramétrico.** Sem isso, n não poderia ser variado sistematicamente.
2. **Representação do grafo unificada em formato CSR** (*Compressed Sparse Row*: um vetor de índices e dois vetores paralelos de destino e peso). As três variantes usam a mesma representação, para que a estrutura da fila de prioridade seja a única variável do experimento — se uma versão usasse lista de adjacência e outra matriz, a diferença de tempo misturaria dois efeitos.
3. **Remoção *lazy* no *heap*.** A operação ATUALIZAR-HEAP do pseudocódigo pressupõe acesso indexado ao interior do *heap*. Como a `priority_queue` da STL não oferece *decrease-key*, inserimos a chave nova e descartamos as entradas obsoletas no momento da extração (teste `d != dist[v]`). Isso mantém a complexidade $O((n + m) \log n)$, apenas com mais entradas na fila.
4. **Buckets circulares.** O pseudocódigo da fonte cria “um bucket para cada distância possível”, o que é inviável aqui: a maior distância pode chegar a $(n - 1) \cdot C$. Usamos a formulação circular de Dial com $C + 1$ *buckets* e índice $\text{dist}[v] \bmod (C + 1)$. Isso é correto porque as chaves presentes na fila nunca diferem mais de C entre si.
5. **Correção de um erro do pseudocódigo da fonte.** Na linha 11 do pseudocódigo do Dijkstra Bucket (Seção 3.3.1 do TCC) consta `P[v] <- v`, o que atribuiria o vértice como predecessor de si mesmo; o correto é `P[u] <- v`, como nas outras duas versões.
6. **Instrumentação de medição** (*harness*) com repetições, saída em CSV e rotina de validação cruzada.

3.4 Validação

Antes de medir tempo, é preciso garantir que as três implementações estão corretas — comparar o tempo de algoritmos que dão respostas diferentes não significaria nada. Para $n = 500$ e $n = 5000$, os vetores de distâncias mínimas produzidos pelas três versões foram comparados posição a posição e resultaram **idênticos**, o que é registrado na saída de erro do programa.

3.5 Metodologia de medição

- **Valores de n :** 1000, 2000, 4000, 8000, 16000, 32000 e 64000 — sete valores dobrando a cada passo, o que permite ler o crescimento diretamente pela razão $T(2n)/T(n)$.
- **Repetições:** 5 execuções independentes por par (algoritmo, n), cada uma sobre um grafo gerado com semente diferente. Reportamos a média e o desvio-padrão.
- **Relógio:** `std::chrono::steady_clock`, resolução de nanossegundos, imune a ajustes do relógio do sistema.
- **O que *não* é medido:** a geração do grafo e a alocação do CSR ocorrem **fora** da região cronometrada, pois não fazem parte do algoritmo. O cronômetro envolve apenas a chamada da função do Dijkstra, incluindo a inicialização de suas estruturas internas (vetor `dist`, `heap` ou `buckets`), que são parte legítima do custo de cada variante.
- **Condições controladas:** mesma máquina, mesmo compilador, mesma flag de otimização (`-O2`) e execução sequencial, sem outras cargas relevantes no sistema.

Tabela 2: Configuração do ambiente de testes.

Item	Descrição
Processador	Intel Xeon @ 2.10 GHz (1 núcleo disponível)
Cache	L1d 48 KiB; L2 2 MiB; L3 260 MiB
Memória (RAM)	3 GB
Sistema operacional	Ubuntu 24.04.4 LTS (64 bits)
Compilador	GNU g++ 13.3.0
Flags de compilação	<code>-O2 -std=c++17</code>
Linguagem	C++17

[PREENCHER: se o grupo reexecutar o experimento em outra máquina — o que é recomendável — substituir os dados da Tabela 2 pelos da máquina utilizada. As tabelas e figuras numéricas são regeradas automaticamente pelo script `codigo/gerar_graficos.py`]

4 Análise teórica da complexidade

Todas as três variantes executam o mesmo esqueleto: cada vértice é fechado exatamente uma vez (portanto n extrações de mínimo) e cada aresta é percorrida no relaxamento (portanto $O(m)$ tentativas de relaxamento, ou $2m$ em grafo não direcionado). A **operação principal**, aquela cujo custo distingue as três versões, é a *extração do mínimo*.

4.1 Dijkstra Normal — $O(n^2)$

Algoritmo 1 DIJKSTRA-NORMAL(G, s)

Entrada: grafo $G = (V, A)$ com pesos não negativos, origem s

Saída: vetores $Dist[]$ e $P[]$

```

1:  $Dist[v] \leftarrow \infty$  e  $P[v] \leftarrow -1$  para todo  $v$ ;    $Dist[s] \leftarrow 0$ 
2:  $L \leftarrow V$ 
3: while  $L \neq \emptyset$  do
4:    $v \leftarrow$  vértice de  $L$  com menor  $Dist[v]$                                  $\triangleright$  varredura linear:  $O(n)$ 
5:   remover  $v$  de  $L$ 
6:   for all  $u$  adjacente a  $v$  do
7:     if  $Dist[u] > Dist[v] + peso(v, u)$  then
8:        $Dist[u] \leftarrow Dist[v] + peso(v, u)$ ;    $P[u] \leftarrow v$ 
9:     end if
10:  end for
11: end while

```

A linha 4 percorre todos os n vértices para achar o menor $Dist$ ainda aberto. Como o laço externo executa n vezes, essa linha sozinha realiza $n \cdot n = n^2$ comparações. Os relaxamentos somam $O(m)$ operações no total. Logo

$$T(n) = \Theta(n^2) + O(m) = \Theta(n^2 + m),$$

e como neste experimento $m = 4n$, o termo n^2 domina: $T(n) = \Theta(n^2)$. A previsão testável é: **dobrar n deve multiplicar o tempo por aproximadamente 4.**

4.2 Dijkstra com Heap Mínimo — $O((n + m) \log n)$

Algoritmo 2 DIJKSTRA-HEAP(G, s)

```

1:  $Dist[v] \leftarrow \infty$  para todo  $v$ ;    $Dist[s] \leftarrow 0$ 
2: inserir  $(0, s)$  no heap mínimo
3: while o heap não está vazio do
4:    $(d, v) \leftarrow$  EXTRAIR-MÍNIMO(heap)                                 $\triangleright O(\log n)$ 
5:   if  $d \neq Dist[v]$  then
6:     continuar                                                             $\triangleright$  entrada obsoleta
7:   end if
8:   for all  $u$  adjacente a  $v$  do
9:     if  $Dist[u] > d + peso(v, u)$  then
10:       $Dist[u] \leftarrow d + peso(v, u)$ ;    $P[u] \leftarrow v$ 
11:      inserir  $(Dist[u], u)$  no heap                                        $\triangleright O(\log n)$ 
12:    end if
13:  end for
14: end while

```

Cada relaxamento bem-sucedido insere um elemento no *heap*, e cada inserção será extraída uma vez. O número total de elementos que passam pelo *heap* é portanto $O(n+m)$, e cada inserção ou extração custa $O(\log(n+m)) = O(\log n)$ para m polinomial em n . Assim $T(n) = O((n+m)\log n)$, que com $m = 4n$ se reduz a $\Theta(n\log n)$. A previsão testável é mais fina que a do caso anterior:

$$\frac{T(2n)}{T(n)} \approx \frac{2n \log_2(2n)}{n \log_2 n} = 2 \left(1 + \frac{1}{\log_2 n}\right),$$

isto é, um valor **ligeiramente acima de 2 e que decresce lentamente conforme n cresce** — cerca de 2,20 em $n = 1000$ e cerca de 2,13 em $n = 32000$.

4.3 Dijkstra com Buckets — $O(m + n + D_{max})$

Algoritmo 3 DIJKSTRA-BUCKET(G, s, C) — *buckets* circulares

```

1:  $Dist[v] \leftarrow \infty$  para todo  $v$ ;    $Dist[s] \leftarrow 0$ 
2: criar  $B = C + 1$  buckets vazios; inserir  $s$  no bucket 0
3:  $d \leftarrow 0$ 
4: while ainda há vértices na fila do
5:   if bucket  $d \bmod B$  está vazio then
6:      $d \leftarrow d + 1$ ; continuar
7:   end if
8:   remover um vértice  $v$  do bucket  $d \bmod B$  ▷  $O(1)$ 
9:   if  $v$  já fechado ou  $Dist[v] \neq d$  then
10:    continuar
11:   end if
12:   for all  $u$  adjacente a  $v$  do
13:     if  $Dist[u] > d + peso(v, u)$  then
14:        $Dist[u] \leftarrow d + peso(v, u)$ ;    $P[u] \leftarrow v$ 
15:       mover  $u$  para o bucket  $Dist[u] \bmod B$  ▷  $O(1)$ 
16:     end if
17:   end for
18: end while

```

Aqui a extração do mínimo custa $O(1)$: como o algoritmo fecha os vértices em ordem não decrescente de distância, basta caminhar com o índice d para frente e nunca voltar. O preço é um termo aditivo: o índice d percorre todos os valores inteiros de 0 até a maior distância final D_{max} , gastando uma verificação por *bucket* vazio encontrado. Logo

$$T(n) = O(m + n + D_{max}).$$

Esse termo D_{max} é a peculiaridade mais importante desta variante: **ele não depende de n** , e sim da magnitude das distâncias, ou seja do diâmetro do grafo multiplicado pela ordem de grandeza dos pesos. Em grafos aleatórios esparsos o diâmetro cresce apenas

como $O(\log n)$, de modo que D_{max} deve ser praticamente constante na faixa testada. A previsão testável é: o tempo deve crescer **linearmente** com n ($T(2n)/T(n) \approx 2$), porém com uma **sobrecarga inicial fixa** que domina para n pequeno — o que deve produzir razões *abaixo* de 2 no início do experimento, aproximando-se de 2 conforme n cresce.

Note também que a corretude desta variante depende de os pesos serem inteiros e limitados por C : com pesos reais ou com C muito grande, a estrutura de *buckets* deixa de ser vantajosa (ou nem é aplicável).

5 Resultados

5.1 Tempos medidos

A Tabela 3 apresenta a média e o desvio-padrão de 5 execuções para cada par (algoritmo, n). Os desvios-padrão são pequenos em relação às médias: o maior desvio relativo observado foi de 8.5%, e ocorreu justamente no menor tamanho de entrada, em que os tempos são de fração de milissegundo e o ruído de medição pesa mais; a partir de $n = 2000$ o maior desvio relativo cai para 6.8%. Isso indica que o número de repetições foi suficiente e que a variação externa durante as medições foi pequena.

Tabela 3: Tempo de execução em milissegundos: média e desvio-padrão de 5 repetições.

n	m	Normal		Heap Mínimo		Bucket	
		média	desvio	média	desvio	média	desvio
1.000	4.000	1.001	0.012	0.255	0.022	0.181	0.003
2.000	8.000	3.759	0.035	0.524	0.013	0.331	0.023
4.000	16.000	14.728	0.130	1.203	0.031	0.558	0.011
8.000	32.000	75.391	0.440	2.674	0.037	1.003	0.030
16.000	64.000	614.846	6.591	5.949	0.199	1.784	0.032
32.000	128.000	2908.010	25.059	13.312	0.518	3.579	0.131
64.000	256.000	12215.186	68.131	28.768	0.465	7.374	0.356

A Figura 1 mostra esses mesmos dados. No eixo x está n , o número de vértices do grafo (com $m = 4n$ arestas); no eixo y , o tempo médio de execução em milissegundos. O painel da esquerda usa escala linear e o da direita escala logarítmica nos dois eixos — em escala log-log, uma função $T(n) = cn^k$ aparece como uma reta de inclinação k , o que permite ler a ordem de crescimento diretamente da figura.

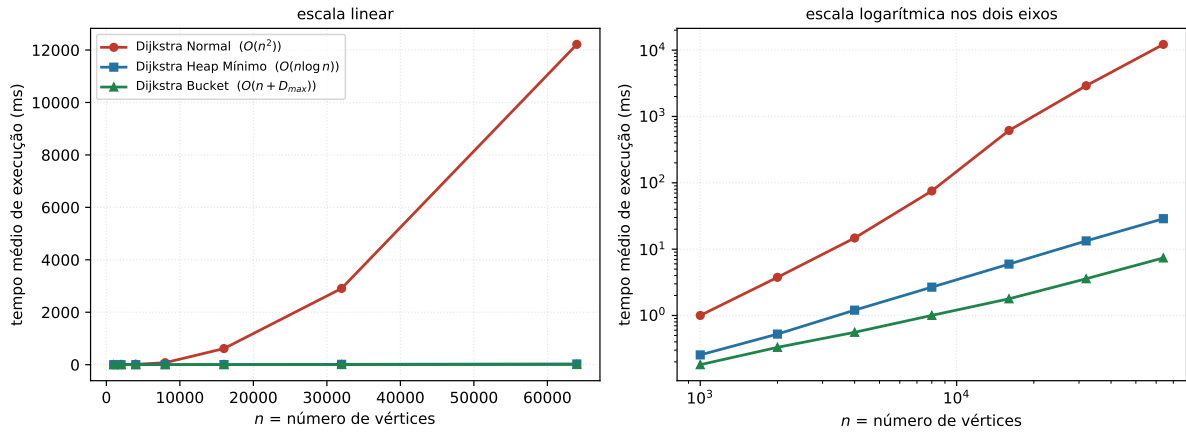


Figura 1: Tempo de execução em função do tamanho da entrada n (gráfico obrigatório 1). Em escala linear (esquerda) o crescimento quadrático do Dijkstra Normal domina de tal forma que as outras duas curvas ficam coladas ao eixo; a escala log-log (direita) separa as três e evidencia inclinações diferentes.

A leitura em escala linear já responde de forma bruta à pergunta central: o Dijkstra Normal sai de 1,0 ms em $n = 1.000$ para 12215 ms em $n = 64.000$, enquanto o Heap Mínimo vai a apenas 28.8 ms e o Bucket a 7.4 ms. No maior tamanho testado, o Heap Mínimo é cerca de 425 vezes mais rápido e o Bucket cerca de 1657 vezes mais rápido que a versão com varredura linear. Vale destacar que essa diferença enorme **não vem de otimização de código**: as três versões compartilham o gerador, a representação do grafo, o compilador e as flags. Vem apenas da estrutura de dados escolhida para a fila de prioridade.

5.2 Comparação com a curva de crescimento esperada

A Figura 2 confronta cada série medida com a sua função de crescimento teórica. A constante multiplicativa de cada curva teórica foi calibrada no menor n medido, ou seja, as curvas partem do mesmo ponto e o que se compara é a *forma* do crescimento, não o valor absoluto — a análise assintótica não prevê constantes.

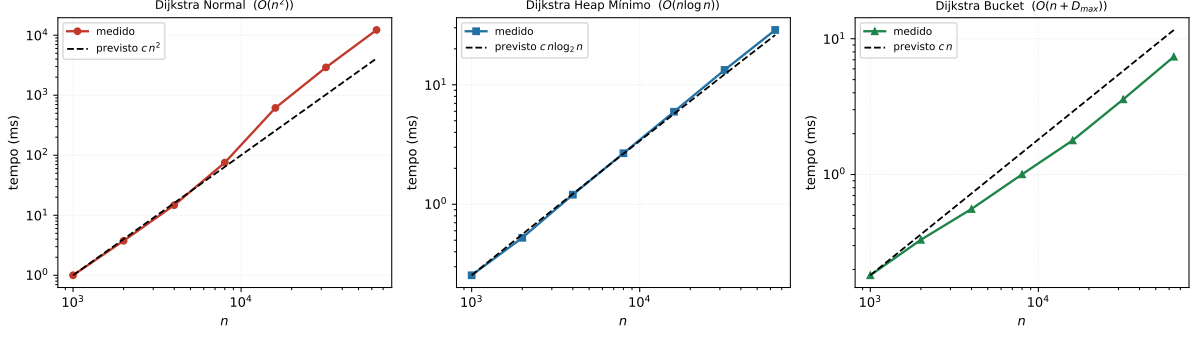


Figura 2: Comportamento empírico *versus* função de crescimento esperada (**gráfico obrigatório 2**), em escala log-log. Linha cheia: medido; linha tracejada: previsto, com a constante ajustada em $n = 1.000$.

Uma forma mais sensível de comparar crescimento, que não depende de calibrar constante alguma, é olhar a razão entre tempos de tamanhos consecutivos. Como n dobra a cada passo, essa razão deve valer aproximadamente 4 para um algoritmo quadrático, um pouco mais de 2 para $n \log n$ e 2 para um algoritmo linear. A Tabela 4 e a Figura 3 apresentam essas razões.

Tabela 4: Razão de crescimento $T(2n)/T(n)$ ao dobrar o tamanho da entrada. Valores de referência: 4 para $O(n^2)$, pouco acima de 2 para $O(n \log n)$, 2 para $O(n)$.

Transição em n	Normal	Heap Mínimo	Bucket
1.000 $\rightarrow$ 2.000	3.75	2.06	1.82
2.000 $\rightarrow$ 4.000	3.92	2.30	1.69
4.000 $\rightarrow$ 8.000	5.12	2.22	1.80
8.000 $\rightarrow$ 16.000	8.16	2.23	1.78
16.000 $\rightarrow$ 32.000	4.73	2.24	2.01
32.000 $\rightarrow$ 64.000	4.20	2.16	2.06

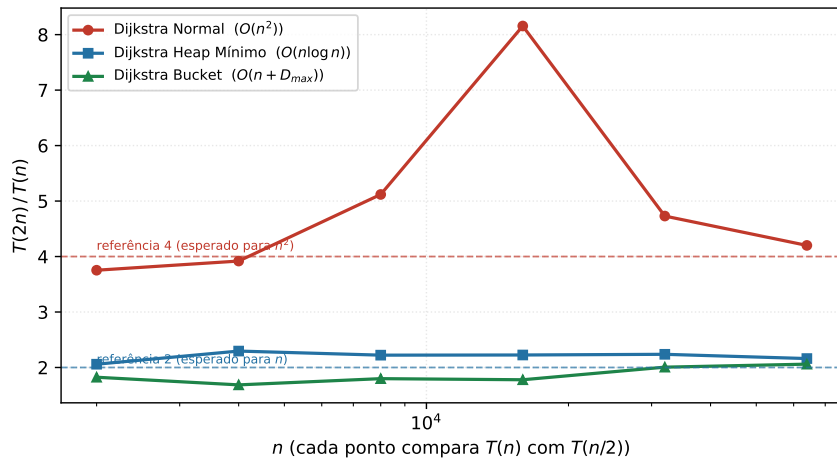


Figura 3: Razões $T(2n)/T(n)$ medidas, com as linhas de referência 4 (quadrático) e 2 (linear).

Complementarmente, ajustando uma reta por mínimos quadrados aos dados em escala log-log obtemos o expoente empírico de cada variante: $k \approx 2.33$ para o Normal, $k \approx 1.15$ para o Heap Mínimo e $k \approx 0.88$ para o Bucket. Os três valores ordenam as variantes exatamente como a teoria prevê (quadrático > levemente superlinear > linear), mas nenhum coincide exatamente com o expoente ideal — é justamente isso que a próxima seção investiga.

5.3 Grandezas estruturais coletadas

Para poder explicar os desvios em vez de apenas constatá-los, coletamos duas grandezas do próprio experimento, fora da medição de tempo (Tabela 5): a maior distância final D_{max} , que limita a varredura de *buckets*, e o tamanho da região de memória varrida a cada iteração do Dijkstra Normal (o vetor `dist` de 8 bytes por vértice mais o vetor de vértices fechados de 1 byte por vértice).

Tabela 5: Grandezas estruturais das instâncias (uma execução por n).

n	D_{max}	<i>Buckets</i> vazios varridos	Memória varrida por iteração (KiB)
1.000	1.737	1.938	9
2.000	1.930	1.991	18
4.000	2.167	2.167	35
8.000	2.232	2.246	70
16.000	2.229	2.229	141
32.000	2.593	2.593	281
64.000	2.615	2.615	562

6 Discussão: teoria $\times$ prática

6.1 O crescimento observado é compatível com a complexidade teórica?

Heap Mínimo: sim, com precisão notável. As razões medidas ficaram entre 2,06 e 2,30, contra a previsão de 2,20 a 2,13 deduzida na Seção 4.2. A constante normalizada $T(n)/(n \log_2 n)$ (Tabela 6) permanece entre 23,9 e 28,2 ns enquanto n cresce 64 vezes — menos de 20% de variação contra um aumento de 6400% na entrada. Na Figura 2, a curva medida e a curva $cn \log_2 n$ são praticamente indistinguíveis. Este é o caso em que teoria e prática coincidiram melhor.

Bucket: sim, mas o comportamento linear só aparece para n grande. As razões começam em 1,82 e 1,69 — abaixo de 2, o que isoladamente pareceria *sublinear*, algo impossível para um algoritmo que precisa examinar todas as $m = 4n$ arestas. A Tabela 5 explica: D_{max} vale 1737 em $n = 1000$ e apenas 2615 em $n = 64000$, ou seja,

crece 1,5 vez enquanto n cresce 64 vezes, confirmando a previsão de que esse termo é quase independente de n . Em $n = 1000$, o algoritmo varre ≈ 1938 índices de *bucket* para processar apenas 1000 vértices: a sobrecarga fixa **domina** o tempo. Conforme n cresce, esse custo constante se dilui, as razões sobem para 2,01 e 2,06, e a constante $T(n)/n$ estabiliza em torno de 112–115 ns. Ou seja, o algoritmo é linear; o que se observava no início era o efeito do termo aditivo D_{max} , exatamente como previsto.

Normal: qualitativamente sim, quantitativamente não. As duas primeiras razões, 3,75 e 3,92, batem com o 4 esperado. Mas as transições seguintes dão 5,12 e, sobretudo, 8,16 — o dobro do previsto — e só depois voltam a 4,73 e 4,20. O expoente empírico global, 2.33, fica bem acima de 2. Há aqui uma divergência real que precisa de explicação.

Tabela 6: Tempo normalizado pela função de crescimento esperada, em nanossegundos. Um valor constante ao longo da coluna indica que a função de crescimento escolhida descreve bem os dados.

n	$T(n)/n^2$ (Normal)	$T(n)/(n \log_2 n)$ (Heap)	$T(n)/n$ (Bucket)
1.000	1.001	25.54	181.3
2.000	0.940	23.90	165.4
4.000	0.920	25.14	139.4
8.000	1.178	25.78	125.4
16.000	2.402	26.63	111.5
32.000	2.840	27.80	111.9
64.000	2.982	28.15	115.2

6.2 Que fatores explicam o desvio do Dijkstra Normal?

A causa não é o algoritmo, e sim a **hierarquia de memória**. A varredura linear do mínimo lê, a cada uma das n iterações, todo o vetor `dist` (8 bytes por vértice) e todo o vetor de vértices fechados (1 byte por vértice) — cerca de $9n$ bytes. A última coluna da Tabela 5 mostra o tamanho dessa região, e a comparação com os caches da máquina (Tabela 2) é reveladora:

- até $n = 4000$, a região varrida ocupa no máximo 35 KiB e cabe inteira na cache L1 de dados (48 KiB). Nessa faixa, $T(n)/n^2 \approx 0,92$ –1,00 ns e as razões batem com 4;
- entre $n = 8000$ e $n = 16000$ a região passa de 70 para 141 KiB e **deixa de caber na L1**, migrando para a L2. É exatamente aí que aparece a razão anômala de 8,16: além do crescimento quadrático de $4\times$, cada acesso passou a custar mais, e os dois efeitos se multiplicaram;
- de $n = 16000$ em diante a região está estabilizada na L2 (que tem 2 MiB, com folga até $n = 64000$) e as razões voltam a 4,73 e 4,20, com $T(n)/n^2$ estabilizando em torno de 2,8–3,0 ns — cerca de três vezes o valor da fase em L1.

Ou seja: o algoritmo é $\Theta(n^2)$ em número de operações, como a teoria prevê, mas o **custo médio de cada operação triplicou** no meio do experimento. A análise assintótica supõe custo unitário para cada acesso à memória; máquinas reais não têm essa propriedade. O desvio observado não refuta a teoria, ele mostra precisamente onde o modelo de custo uniforme deixa de valer.

6.3 Em que ponto a diferença entre os algoritmos fica evidente?

Já em $n = 1000$ o Normal é 4 vezes mais lento que o Heap e 5,5 vezes mais lento que o Bucket. A diferença é pequena em termos absolutos (1 ms contra 0,2 ms) e, para uma aplicação que roda o algoritmo uma vez sobre um grafo pequeno, escolher a estrutura “errada” seria irrelevante. O quadro muda de natureza com a escala: em $n = 64.000$ a diferença é de 12,2 segundos contra 29 milissegundos. É o tipo de diferença que separa um sistema utilizável de um sistema inviável — e é a razão prática de a análise assintótica importar.

Entre as duas variantes eficientes, o **Bucket foi consistentemente mais rápido que o Heap Mínimo** em toda a faixa testada, sendo cerca de 3.9 vezes mais rápido em $n = 64.000$. Isso *contraria* a conclusão da fonte principal, que encontrou o Heap Mínimo cerca de seis vezes mais rápido que o Bucket. A divergência é explicável e vale a pena examiná-la, porque ilustra bem o papel das características da entrada: o custo do Bucket carrega o termo D_{max} , proporcional às magnitudes das distâncias. Nas instâncias do DIMACS usadas pela fonte, os pesos são distâncias e tempos de viagem reais em redes de estradas, com valores muito grandes e caminhos longos, o que torna D_{max} enorme e penaliza severamente o Bucket. Nas nossas instâncias, com pesos em $[1, 1000]$ e diâmetro pequeno, D_{max} ficou em torno de 2600 e a vantagem do acesso $O(1)$ prevaleceu. Nenhum dos dois resultados está errado: eles medem regimes de entrada diferentes, e a lição é que a escolha da estrutura depende do perfil dos pesos, não apenas da ordem assintótica.

6.4 O que acontece quando n aumenta muito?

A extrapolação a partir das constantes estabilizadas é direta. Para $n = 10^6$ com a mesma densidade, o Normal levaria aproximadamente $3,0 \times (10^6)^2$ ns ≈ 50 minutos, enquanto o Heap levaria cerca de $28 \times 10^6 \times 20$ ns $\approx 0,6$ s e o Bucket cerca de 115×10^6 ns $\approx 0,1$ s. Essa é a diferença entre uma consulta interativa e um processamento em lote. A ressalva importante é que essa extrapolação supõe que as constantes permaneçam estáveis, e a própria Seção 6.2 mostrou que elas podem mudar quando o conjunto de dados atravessa um nível da hierarquia de memória.

7 Conclusão

O experimento respondeu à pergunta central: o tempo de execução do algoritmo de Dijkstra cresce de formas qualitativamente diferentes segundo a estrutura de dados da fila de prioridade, e as três ordens de crescimento observadas são compatíveis com as previsões assintóticas obtidas da análise das operações.

As principais descobertas foram:

1. A variante com **Heap Mínimo** reproduziu $O(n \log n)$ com precisão notável: razões $T(2n)/T(n)$ entre 2,06 e 2,30 e constante normalizada praticamente estável em uma faixa de 64 vezes em n .
2. A variante com **Buckets** é linear, mas seu comportamento assintótico só se manifesta depois que n supera a sobrecarga fixa D_{max} . Medir essa grandeza diretamente (que ficou entre 1737 e 2615 enquanto n crescia 64 vezes) permitiu explicar as razões aparentemente sublineares em vez de tratá-las como ruído.
3. A variante **Normal** é $\Theta(n^2)$ em operações, mas seu tempo cresceu mais rápido que n^2 na faixa intermediária. A causa foi identificada: a região de memória varrida a cada iteração deixa de caber na cache L1 entre $n = 8000$ e $n = 16000$, e o custo médio por operação triplica. Esse é o exemplo mais didático que o trabalho produziu de por que “análise assintótica descreve o crescimento, mas o tempo medido depende da implementação e do ambiente”.
4. A comparação com a fonte principal levou a uma **conclusão oposta** quanto à disputa Heap *versus* Bucket. Isso não é contradição: a fonte usou redes de estradas reais, com distâncias de magnitude muito maior, regime no qual o termo D_{max} do Bucket pesa. A ordem assintótica não basta para escolher uma estrutura; o perfil da entrada importa.

Limitações

- Os grafos são **aleatórios com densidade fixa** ($m = 4n$). Grafos reais têm estrutura (comunidades, distribuição de graus desigual, planaridade) que afeta tanto o diâmetro quanto a localidade de acesso à memória. Repetir o experimento com as instâncias do DIMACS usadas pela fonte permitiria comparação direta.
- Só a **densidade esparsa** foi testada. Em grafos densos ($m \approx n^2$), a vantagem do *heap* diminui e o Dijkstra Normal volta a ser competitivo, pois seu termo $O(n^2)$ deixa de ser dominante.

- O intervalo de pesos foi fixado em $C = 1000$. Como a discussão mostrou, C influencia diretamente o Bucket; variar C mantendo n fixo seria um segundo experimento natural.
- As medições vêm de uma **única máquina** e de um único compilador. A faixa de n vai até 64000, limitada pelo tempo quadrático da variante Normal.
- Mediu-se apenas **tempo**, não consumo de memória — que é justamente o ponto fraco reconhecido do *heap* e dos *buckets* frente à versão simples.

Declaração de uso de ferramentas de IA

Em atendimento à exigência de declarar o uso de IA em todas as etapas do trabalho, registramos:

[PREENCHER: revisar e ajustar esta declaração para refletir exatamente o que o grupo fez — o que segue descreve o uso ocorrido na elaboração deste material]

- **Busca da fonte:** **[PREENCHER: indicar se houve uso de IA nesta etapa]**
- **Implementação:** os pseudocódigos das três variantes foram obtidos do TCC citado; a transcrição para C++, o gerador de grafos, o *harness* de medição e a rotina de validação foram desenvolvidos com auxílio do assistente Claude (Anthropic), com revisão e verificação pelo grupo. A adaptação dos *buckets* para a forma circular e a correção da linha 11 do pseudocódigo da fonte surgiram dessa revisão.
- **Execução do experimento:** as medições foram executadas pelo próprio código, sem intervenção de IA nos dados. Os valores das tabelas e figuras são gerados automaticamente a partir do arquivo `resultados/resultados_brutos.csv` pelo script `codigo/gerar_graficos.py`.
- **Análise e redação:** a estruturação do relatório, a redação e a interpretação dos resultados contaram com auxílio do mesmo assistente, a partir dos dados efetivamente medidos, com revisão do grupo.
- **Nenhum dado experimental foi gerado ou estimado por IA.**

Referências

Referências

- [1] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos: Teoria e Prática**. 3. ed. Rio de Janeiro: Elsevier, 2009.

- [2] DIAL, R. B. Algorithm 360: Shortest-path forest with topological ordering. **Communications of the ACM**, v. 12, n. 11, p. 632–633, 1969.
- [3] DIJKSTRA, E. W. A note on two problems in connexion with graphs. **Numerische Mathematik**, v. 1, p. 269–271, 1959.
- [4] DIMACS. **9th DIMACS Implementation Challenge — Shortest Paths**. Disponível em: <https://www.diag.uniroma1.it/challenge9/download.shtml>. Acesso em: **[PRE-ENCHER: data]**.
- [5] RODRIGUES, G. C. S. **Análise de desempenho do algoritmo de Dijkstra com diferentes estruturas de dados**. 2023. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação) — Universidade Federal do Espírito Santo, Alegre, 2023. Código-fonte disponível em: <https://github.com/guiconrado/Djkstra>.
- [6] SEDGEWICK, R.; WAYNE, K. **Algorithms**. 4. ed. Boston: Addison-Wesley Professional, 2011.
- [7] TARJAN, R. E. **Data Structures and Network Algorithms**. Philadelphia: Society for Industrial and Applied Mathematics, 1983.

A Instruções de execução

Estrutura dos arquivos entregues:

<code>main.tex</code>	este relatorio
<code>codigo/dijkstra_experimento.cpp</code>	implementacao das 3 variantes
<code>codigo/gerar_graficos.py</code>	agregacao, figuras e tabelas
<code>resultados/resultados_brutos.csv</code>	todas as medicoes individuais
<code>resultados/estatisticas_estruturais.csv</code>	Dmax e varreduras de bucket
<code>resultados/tabelas.tex</code>	tabelas geradas (incluido no .tex)
<code>resultados/log_execucao.txt</code>	log, com o resultado da validacao
<code>figuras/*.pdf</code>	graficos gerados

Para reproduzir o experimento inteiro:

```
cd codigo
g++ -O2 -std=c++17 -o dijkstra dijkstra_experimento.cpp
./dijkstra > ../resultados/resultados_brutos.csv \
    2> ../resultados/log_execucao.txt
./dijkstra stats > ../resultados/estatisticas_estruturais.csv
python3 gerar_graficos.py          # requer matplotlib
cd .. && pdflatex main.tex && pdflatex main.tex
```

Como as tabelas e figuras do relatório são geradas a partir dos CSV, basta reexecutar esses comandos em outra máquina para que todos os números do documento sejam atualizados. Tempo total de execução na máquina da Tabela 2: cerca de 1 minuto e 30 segundos, dominado pela variante Normal em $n = 64000$.

Repositório do grupo: **[PREENCHER: link do repositório, se houver]**

B Código-fonte

```
1 // =====
2 // Analise Empirica de Algoritmos - Estrutura de Dados Basicas II
3 // Algoritmo de Dijkstra com tres estruturas de dados distintas:
4 // (1) Dijkstra Normal - busca linear do minimo ->  $O(V^2 + A)$ 
5 // (2) Dijkstra Heap Min - fila de prioridade (heap bin.) ->  $O((V + A) \log V)$ 
6 // (3) Dijkstra Bucket - buckets circulares (Dial 1969) ->  $O(A + V + C)$ 
7 //
8 // Os pseudocodigos das tres versoes foram reproduzidos de:
9 // RODRIGUES, G. C. S. Analise de desempenho do algoritmo de Dijkstra
10 // com diferentes estruturas de dados. TCC, UFES, Alegre-ES, 2023.
11 // Codigo original do TCC: https://github.com/guiconrado/Dijkstra
12 //
13 // Adaptacoes feitas por este grupo (ver Secao 3 do relatorio):
14 // - substituicao das instancias fixas do DIMACS por um gerador
15 // parametrico de grafos conexos, permitindo variar n de forma
16 // controlada;
17 // - insercao do harness de medicao de tempo (chrono) com repeticoes;
18 // - insercao da rotina de validacao cruzada entre as tres versoes;
19 // - saida em CSV para posterior geracao de graficos.
20 //
21 // Compilacao: g++ -O2 -std=c++17 -o dijkstra dijkstra_experimento.cpp
22 // Execuciao: ./dijkstra > resultados_brutos.csv
23 // =====
24
25 #include <bits/stdc++.h>
26 using namespace std;
27 using Clock = chrono::steady_clock;
28
29 const long long INF = numeric_limits<long long>::max() / 4;
30
31 // -----
32 // Representacao do grafo em formato CSR (Compressed Sparse Row).
33 // Escolhida por ser compacta e por dar localidade de memoria igual
34 // para as tres versoes do algoritmo - assim a estrutura de dados da
35 // fila de prioridade e a UNICA variavel do experimento.
36 // -----
37 struct Grafo {
38     int n; // numero de vertices (este e o nosso "n")
39     long long m; // numero de arestas (nao direcionadas)
40     vector<int> ini; // ini[v] .. ini[v+1]-1 : faixa de v em dest/peso
41     vector<int> dest; // vertice de destino da aresta
42     vector<int> peso; // peso/custo da aresta
43     int C; // maior peso de aresta (necessario ao Bucket)
44 };
45
46 // -----
47 // Gerador de grafo conexo, nao direcionado e com pesos inteiros em [1,C].
48 // 1) sorteia uma arvore geradora aleatoria (garante conexidade);
49 // 2) completa com arestas aleatorias ate atingir m arestas.
50 // Manter  $m = \text{grau\_medio}/2 * n$  mantem a DENSIDADE fixa, de modo que o
51 // unico parametro que cresce e n.
52 // -----
53 Grafo gerarGrafo(int n, int grauMedio, int C, uint64_t semente) {
54     mt19937_64 rng(semente);
55     uniform_int_distribution<int> distPeso(1, C);
56 }
```

```

57     long long m = (long long)n * grauMedio / 2;
58     if (m < n - 1) m = n - 1;
59
60     vector<int> u, v, w;
61     u.reserve(m); v.reserve(m); w.reserve(m);
62
63     // (1) arvore geradora aleatoria
64     for (int i = 1; i < n; ++i) {
65         int pai = (int)(rng() % (uint64_t)i);
66         u.push_back(pai); v.push_back(i); w.push_back(distPeso(rng));
67     }
68     // (2) arestas extras
69     while ((long long)u.size() < m) {
70         int a = (int)(rng() % (uint64_t)n);
71         int b = (int)(rng() % (uint64_t)n);
72         if (a == b) continue;
73         u.push_back(a); v.push_back(b); w.push_back(distPeso(rng));
74     }
75
76     Grafo g;
77     g.n = n;
78     g.m = (long long)u.size();
79     g.C = C;
80     g.ini.assign(n + 1, 0);
81     for (size_t i = 0; i < u.size(); ++i) { g.ini[u[i] + 1]++; g.ini[v[i] + 1]++; }
82     for (int i = 0; i < n; ++i) g.ini[i + 1] += g.ini[i];
83     g.dest.resize(2 * u.size());
84     g.peso.resize(2 * u.size());
85     vector<int> pos(g.ini.begin(), g.ini.end() - 1);
86     for (size_t i = 0; i < u.size(); ++i) {
87         g.dest[pos[u[i]]] = v[i]; g.peso[pos[u[i]]++] = w[i];
88         g.dest[pos[v[i]]] = u[i]; g.peso[pos[v[i]]++] = w[i];
89     }
90     return g;
91 }
92
93 // =====
94 // (1) DIJKSTRA NORMAL - selecao do minimo por varredura linear do vetor
95 //     Cada uma das n iteracoes varre os n vertices em busca do menor
96 //     Dist[] ainda aberto -> n^2 operacoes de comparacao.
97 // =====
98 void dijkstraNormal(const Grafo& g, int s, vector<long long>& dist) {
99     int n = g.n;
100    dist.assign(n, INF);
101    vector<char> fechado(n, 0);
102    dist[s] = 0;
103
104    for (int it = 0; it < n; ++it) {
105        // --- extrai o minimo: O(n) ---
106        int v = -1;
107        long long melhor = INF;
108        for (int k = 0; k < n; ++k)
109            if (!fechado[k] && dist[k] < melhor) { melhor = dist[k]; v = k; }
110        if (v < 0) break; // restaram apenas inalcançaveis
111        fechado[v] = 1;
112        // --- relaxamento das arestas de v ---
113        for (int e = g.ini[v]; e < g.ini[v + 1]; ++e) {
114            int u = g.dest[e];
115            if (dist[v] + g.peso[e] < dist[u]) dist[u] = dist[v] + g.peso[e];

```

```

116     }
117 }
118 }
119
120 // =====
121 // (2) DIJKSTRA COM HEAP MINIMO - fila de prioridade com remocao lazy
122 //     Extrair-Minimo e Atualizar-Heap custam  $O(\log n)$ .
123 // =====
124 void dijkstraHeap(const Grafo& g, int s, vector<long long>& dist) {
125     int n = g.n;
126     dist.assign(n, INF);
127     priority_queue<pair<long long,int>,
128                 vector<pair<long long,int>>,
129                 greater<pair<long long,int>>> heap;
130     dist[s] = 0;
131     heap.push({0, s});
132
133     while (!heap.empty()) {
134         auto [d, v] = heap.top(); heap.pop();
135         if (d != dist[v]) continue; // entrada obsoleta (lazy delete)
136         for (int e = g.ini[v]; e < g.ini[v + 1]; ++e) {
137             int u = g.dest[e];
138             long long nd = d + g.peso[e];
139             if (nd < dist[u]) { dist[u] = nd; heap.push({nd, u}); }
140         }
141     }
142 }
143
144 // =====
145 // (3) DIJKSTRA COM BUCKETS (Dial) - fila de prioridade monotona
146 //     Como todo peso esta em  $[1, C]$ , as chaves presentes na fila nunca
147 //     diferem mais de  $C$  entre si; logo  $C+1$  buckets circulares bastam.
148 //     Extracao e insercao sao  $O(1)$ ; o custo extra vem da varredura
149 //     sequencial dos indices de bucket, proporcional a maior distancia.
150 // =====
151 void dijkstraBucket(const Grafo& g, int s, vector<long long>& dist) {
152     int n = g.n;
153     int B = g.C + 1; // numero de buckets circulares
154     dist.assign(n, INF);
155     vector<char> fechado(n, 0);
156     vector<vector<int>> bucket(B);
157
158     dist[s] = 0;
159     bucket[0].push_back(s);
160     long long naFila = 1;
161     long long d = 0;
162
163     while (naFila > 0) {
164         int idx = (int)(d % B);
165         if (bucket[idx].empty()) { ++d; continue; } // pula bucket vazio
166         int v = bucket[idx].back(); bucket[idx].pop_back(); --naFila;
167         if (fechado[v] || dist[v] != d) continue; // entrada obsoleta
168         fechado[v] = 1;
169         for (int e = g.ini[v]; e < g.ini[v + 1]; ++e) {
170             int u = g.dest[e];
171             long long nd = d + g.peso[e];
172             if (nd < dist[u]) {
173                 dist[u] = nd;
174                 bucket[(int)(nd % B)].push_back(u);

```

```

175         ++naFila;
176     }
177 }
178 }
179 }
180
181 // -----
182 // Validacao: as tres versoes devem devolver EXATAMENTE o mesmo vetor
183 // de distancias minimas. Sem isso, comparar tempos nao faria sentido.
184 // -----
185 bool validar(int n, int grauMedio, int C, uint64_t semente) {
186     Grafo g = gerarGrafo(n, grauMedio, C, semente);
187     vector<long long> a, b, c;
188     dijkstraNormal(g, 0, a);
189     dijkstraHeap(g, 0, b);
190     dijkstraBucket(g, 0, c);
191     return a == b && b == c;
192 }
193
194 // -----
195 // Modo "stats": coleta grandezas estruturais que ajudam a explicar os
196 // tempos observados - a maior distancia final (Dmax, que limita a
197 // varredura de buckets) e o numero de indices de bucket vazios
198 // visitados. NAO faz parte da medicao de tempo.
199 // -----
200 void estatisticasBucket(const Grafo& g, int s, long long& dmax, long long& varreduras) {
201     int n = g.n, B = g.C + 1;
202     vector<long long> dist(n, INF);
203     vector<char> fechado(n, 0);
204     vector<vector<int>> bucket(B);
205     dist[s] = 0; bucket[0].push_back(s);
206     long long naFila = 1, d = 0;
207     varreduras = 0;
208     while (naFila > 0) {
209         int idx = (int)(d % B);
210         if (bucket[idx].empty()) { ++d; ++varreduras; continue; }
211         int v = bucket[idx].back(); bucket[idx].pop_back(); --naFila;
212         if (fechado[v] || dist[v] != d) continue;
213         fechado[v] = 1;
214         for (int e = g.ini[v]; e < g.ini[v + 1]; ++e) {
215             int u = g.dest[e];
216             long long nd = d + g.peso[e];
217             if (nd < dist[u]) { dist[u] = nd; bucket[(int)(nd % B)].push_back(u); ++
                naFila; }
218         }
219     }
220     dmax = 0;
221     for (int v = 0; v < n; ++v) if (dist[v] < INF) dmax = max(dmax, dist[v]);
222 }
223
224 int main(int argc, char** argv) {
225     const int GRAU_MEDIO = 8; // densidade fixa: m = 4n arestas
226     const int C = 1000; // pesos inteiros em [1, 1000]
227     const int REPETICOES = 5;
228     vector<int> tamanhos = {1000, 2000, 4000, 8000, 16000, 32000, 64000};
229
230     // ----- modo estatisticas estruturais -----
231     if (argc > 1 && string(argv[1]) == "stats") {
232         printf("n,m,dmax,varreduras_bucket\n");

```

```

233     for (int n : tamanhos) {
234         Grafo g = gerarGrafo(n, GRAU_MEDIO, C, 12345ULL + 1000ULL * n + 1);
235         long long dmax, varr;
236         estatisticasBucket(g, 0, dmax, varr);
237         printf("%d,%lld,%lld,%lld\n", n, g.m, dmax, varr);
238     }
239     return 0;
240 }
241
242 // ----- etapa de validacao -----
243 for (int n : {500, 5000}) {
244     bool ok = validar(n, GRAU_MEDIO, C, 987654321ULL + n);
245     fprintf(stderr, "[validacao] n=%d -> %s\n", n, ok ? "OK (3 versoes concordam)" :
246         "FALHOU");
247     if (!ok) return 1;
248 }
249
250 // ----- etapa de medicacao -----
251 printf("n,m,grau_medio,C,algoritmo,repeticao,tempo_ms\n");
252 for (int n : tamanhos) {
253     for (int r = 1; r <= REPETICOES; ++r) {
254         // o grafo e gerado FORA da medicacao (nao faz parte do algoritmo)
255         Grafo g = gerarGrafo(n, GRAU_MEDIO, C, 12345ULL + 1000ULL * n + r);
256         vector<long long> dist;
257
258         struct Alvo { const char* nome; void (*fn)(const Grafo&, int, vector<long
259             long>&); };
260         Alvo alvos[3] = {
261             {"Dijkstra Normal", dijkstraNormal},
262             {"Dijkstra Heap Minimo", dijkstraHeap},
263             {"Dijkstra Bucket", dijkstraBucket}
264         };
265         for (auto& alvo : alvos) {
266             auto t0 = Clock::now();
267             alvo.fn(g, 0, dist);
268             auto t1 = Clock::now();
269             double ms = chrono::duration<double, milli>(t1 - t0).count();
270             printf("%d,%lld,%d,%d,%s,%d,%.6f\n",
271                 n, g.m, GRAU_MEDIO, C, alvo.nome, r, ms);
272         }
273         fprintf(stderr, "n=%d rep=%d concluido\n", n, r);
274     }
275 }
276 return 0;
277 }

```

Listing 1: codigo/dijkstra_experimento.cpp